

Introduction To Programming Using C++ Level 2

Add Elements

#29_Add_Elements



Introduction To Programming Using C++

Level 2

Add Elements

Definition

- ◆ In C++, we can add elements to a `Vector` during Run Time using the `push\_back()` function.
- ◆ `push\_back()` always adds the new element to the end of the Vector.
- ◆ Each call to `push\_back()` increases the Vector's number of elements by one.
- ◆ A Vector automatically manages its storage and can grow when more elements are added.

Explanation

`push\_back()` Function

-  `push\_back()` is a member function of `std::vector`.

-  It is used to add an element to the end of the Vector.

-  Syntax:

 - `vNumbers.push_back(value);`

-  The new element is placed after the current last element.

-  We do not need to manually specify the position where the new element should be added.

-  If the Vector needs more storage, it automatically allocates additional memory.

💡 Example

```
int main()
{
    vector<int> vNumbers;
    vNumbers.push_back(10);
    vNumbers.push_back(20);
    vNumbers.push_back(30);
    vNumbers.push_back(40);
    vNumbers.push_back(50);
    cout << "Numbers Vector:\n";
    for (int &Number : vNumbers)
    {
        cout << Number << endl;
    }
    return 0;
}
```

➡ Result

Numbers Vector:

10

20

30

40

50

Note

- ◆ `push_back()` does not replace existing elements. It adds a new element to the end.
- ◆ You can call `push_back()` multiple times to add multiple elements.
- ◆ `push_back()` is especially useful inside loops when receiving data from the user or processing data dynamically.
- ◆ When simply displaying elements, using a Reference can avoid unnecessary copying:

```
for (const int &Number : vNumbers)
{
    cout << Number << " ";
}
```
- ◆ For small types such as `int`, copying is inexpensive. For larger objects, using a `const` Reference can be more beneficial.

⊘ Warning

- Do not confuse `push_back()` with `insert()`.
- ▨ `push_back()` adds an element specifically to the end of the Vector.
- ▤ `insert()` can add an element at a specific position inside the Vector.
- Vector storage is managed automatically. You normally do not need to manually allocate memory before using `push_back()`.



Comparison

Vector

- ✓ `push_back()` adds an element to the end.
- ✓ `pop_back()` removes the last element.
- ✓ A Vector can access any element using its index.
- ✓ A Vector is a dynamic container.

Stack

- ✓ `push()` adds an element to the top.
- ✓ `pop()` removes the top element.
- ✓ A Stack follows the LIFO principle.
- ✓ LIFO = Last In, First Out.

✦ Summary

- ✓ `push_back()` is used to add elements to a Vector.
- ✓ Each call adds one new element to the end.
- ✓ A Vector automatically manages its storage as it grows.
- ✓ `push_back()` is commonly used inside loops and during user input.
- ✓ `pop_back()` removes the last element.
- ✓ A Vector can be used to implement LIFO behavior when using `push_back()` and `pop_back()` together.
- ✓ A Vector remains a flexible dynamic container and is not limited to Stack behavior.

Thank you 😊